Practical NO.09

Name - Sakshi Vijaykumar Muthiyan

Roll No. -

Title - Data Wrangling

Problem Statement: Data Wrangling on Real Estate Market

Dataset: "RealEstate_Prices.csv"

Description: The dataset contains information about housing prices in a specific real estate market. It includes various attributes such as property characteristics, location, sale prices, and other relevant features. The goal is to perform data wrangling to gain insights into the factors influencing housing prices and prepare the dataset for further analysis or modeling.

In [3]:
```
import pandas as pd
import numpy as np
# path to the local practical files inside desktop folder
path_main = r"C:\Users\muthi\OneDrive\Desktop\PRACTICAL EXAM\ML\RealEstat
print("Loading real estate market records...")
try:
    df = pd.read_csv(path_main)
except FileNotFoundError:
    # file generator
    print("Files missing. Generating target manual CSV data records autom
    np.random.seed(42)
    main_data = {
        'Property  ID ': [f'PROP-{i}' for i in range(1, 101)],
        'Neighborhood Name!': ['Kothrud', 'Baner', 'Hinjewadi', 'Wakad',
        'Sale Price ($)': np.random.uniform(60000, 240000, 100),
        'Bedrooms_Qty': np.random.randint(1, 5, 100).astype(float),
        'Year Built': np.random.randint(1995, 2024, 100)
    }
    df = pd.DataFrame(main_data)
    df.loc[3, 'Bedrooms_Qty'] = np.nan # injecting missing row for task 2
    df.loc[8, 'Sale Price ($)'] = 9500000.0 # injecting extreme outlier f
    df.to_csv(path_main, index=False)
# task 1: strip whitespace, remove special symbols, adjust character form
print("\nColumn headers before cleaning string characters:", list(df.colu
df.columns = df.columns.str.strip().str.replace('[!@#$%^&*()$ ]', '', reg
print("Column headers after text cleaning complete:", list(df.columns))
df.head(3)
```

```
Loading real estate market records...

Column headers before cleaning string characters: ['Property  ID ', 'Neig
hborhood Name!', 'Sale Price ($)', 'Bedrooms_Qty', 'Year Built']
Column headers after text cleaning complete: ['PropertyID', 'Neighborhood
Name', 'SalePrice', 'Bedrooms_Qty', 'YearBuilt']
```

Out[3]:

	PropertyID	NeighborhoodName	SalePrice	Bedrooms_Qty	YearBuilt
0	PROP-1	Kothrud	127417.221393	3.0	1995
1	PROP-2	Baner	231128.575154	4.0	1995
2	PROP-3	Hinjewadi	191758.909526	3.0	2013

In [4]:

```
# task 2: verify and resolve missing values
print("=== Task 2: Checking Null Values Before Imputation ===")
print(df.isnull().sum())
df['Bedrooms_Qty'] = df['Bedrooms_Qty'].fillna(df['Bedrooms_Qty'].median(
print("Total empty tracking rows remaining:", df.isnull().sum().sum())
# task 3: merge secondary metadata table to complete SQL-style dataset jo
print("\n=== Task 3: Merging Dataset Sheets ===")
path_meta = r"C:\Users\muthi\OneDrive\Desktop\PRACTICAL EXAM\ML\Neighborh
try:
    df_meta = pd.read_csv(path_meta)
except FileNotFoundError:
    meta_data = {
        'NeighborhoodName': ['Kothrud', 'Baner', 'Hinjewadi', 'Wakad', 'H
        'SafetyRating': [4.7, 4.4, 4.1, 4.2, 4.0],
        'HasMetro': ['Yes', 'Yes', 'No', 'No', 'No']
    }
    df_meta = pd.DataFrame(meta_data)
    df_meta.to_csv(path_meta, index=False)
# executing inner merge join across the clean neighborhood keys matching
df_merged = pd.merge(df, df_meta, left_on='NeighborhoodName', right_on='N
print("Merged database shape matrix layout dimensions:", df_merged.shape)
# printing 2 rows
print("\nPreviewing first 2 rows of merged database matrix:")
print(df_merged.head(2))
```

```
=== Task 2: Checking Null Values Before Imputation ===
PropertyID          0
NeighborhoodName    0
SalePrice           0
Bedrooms_Qty        1
YearBuilt           0
dtype: int64
Total empty tracking rows remaining: 0

=== Task 3: Merging Dataset Sheets ===
Merged database shape matrix layout dimensions: (100, 7)

Previewing first 2 rows of merged database matrix:
  PropertyID NeighborhoodName      SalePrice  Bedrooms_Qty  YearBuilt  \
0     PROP-1          Kothrud  127417.221393           3.0       1995
1     PROP-2            Baner  231128.575154           4.0       1995

   SafetyRating HasMetro
0           4.7      Yes
1           4.4      Yes
```

In [6]:
```
# task 4: filter properties built since year 2010 with target price limit
print("=== Task 4: Subsetting and Filtering Data ===")
print("Row count before criteria slice filter:", df_merged.shape)
# Added .copy() at the end to declare an independent dataframe
df_filtered = df_merged[(df_merged['YearBuilt'] >= 2010) & (df_merged['Sa
print("Row count after filtering criteria slice:", df_filtered.shape)
# task 5: handle strings using categorical code mappings
print("\n=== Task 5: Encoding Categorical Variables ===")
df_filtered['Neighborhood_Code'] = df_filtered['NeighborhoodName'].astype
print("Encoded categories vector mapping preview check:")
print(df_filtered[['NeighborhoodName', 'Neighborhood_Code']].head(4))
```

```
=== Task 4: Subsetting and Filtering Data ===
Row count before criteria slice filter: (100, 7)
Row count after filtering criteria slice: (55, 7)

=== Task 5: Encoding Categorical Variables ===
Encoded categories vector mapping preview check:
  NeighborhoodName  Neighborhood_Code
2        Hinjewadi                  2
4         Hadapsar                  1
6            Baner                  0
8            Wakad                  4
```

In [7]:
```
# task 6: grouping rows to calculate derived group statistics
print("=== Task 6: Data Grouping and Summary Aggregations ===")
summary_report = df_filtered.groupby('NeighborhoodName').agg({
    'SalePrice': ['mean', 'max', 'min'],
    'SafetyRating': 'mean',
    'PropertyID': 'count'
}).reset_index()
# flattening the nested column naming text index
summary_report.columns = ['Neighborhood', 'Avg_Price', 'Max_Price', 'Min_
# layout floating points
pd.set_option('display.float_format', lambda x: '%.2f' % x)
print(summary_report)
```

```
=== Task 6: Data Grouping and Summary Aggregations ===
  Neighborhood  Avg_Price  Max_Price  Min_Price  Safety_Score  Property_C
ount
0        Baner  123857.53  191329.11   70455.05          4.40
12
1     Hadapsar  131217.74  233813.77   79420.46          4.00
11
2    Hinjewadi  182059.97  229109.81   77580.98          4.10
7
3      Kothrud  159840.29  225937.36   63705.21          4.70
11
4        Wakad  823256.89 9500000.00   64575.44          4.20
14
```

In [8]:
```
# task 7: identify and drop extreme outliers using box plot whiskers
print("=== Task 7: Outlier Detection and Matrix Trimming ===")
print("Highest property price value before trimming:", df_filtered['SaleP
```

```python
q1 = df_filtered['SalePrice'].quantile(0.25)
q3 = df_filtered['SalePrice'].quantile(0.75)
iqr = q3 - q1
upper_fence = q3 + 1.5 * iqr
lower_fence = q1 - 1.5 * iqr
# cap the dataset array keeping objects inside valid fences
df_clean = df_filtered[(df_filtered['SalePrice'] >= lower_fence) & (df_fi
print("Highest property price value after outlier trimming:", df_clean['S
print(f"Final data matrix shape dimensions ready for future analysis: {df
```

```
=== Task 7: Outlier Detection and Matrix Trimming ===
Highest property price value before trimming: 9500000.0
Highest property price value after outlier trimming: 233813.7659534207
Final data matrix shape dimensions ready for future analysis: (54, 8)
```